SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: ITA 0605 COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO3: Bayes Theorem – Concept Learning – Maximum Likelihood – Minimum Description Length Principle – Bayes Optimal Classifier – Gibbs Algorithm – Naïve Bayes Classifier – Bayesian Belief Network – EM Algorithm – Probability Learning – Sample Complexity – Finite and Infinite Hypothesis Spaces – Mistake Bound Model,
(BL 6)

Assessment Tool Weightage: 40%

QUESTIONS: 15 **Total Marks:25**

Annexure A

Experiments

Code of Conduct:

*I **Mathiprakash E 192312085** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

1. Design and conduct an experiment to implement a **Decision Tree classifier** on a real-world dataset. Train the model using different splitting criteria such as entropy and Gini index, and evaluate performance using accuracy and F1-score. Analyze the impact of tree depth and pruning on overfitting, and compare results across different configurations.

Aim

To implement a Decision Tree classifier using **Gini Index and Entropy**, evaluate accuracy and F1-score, and study the effect of tree depth and pruning on overfitting.

Dataset

Breast Cancer Wisconsin Dataset from `sklearn`, containing 569 samples and 30 features.

Program

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, f1_score

data = load_breast_cancer()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

for criterion in ["gini", "entropy"]:
    model = DecisionTreeClassifier(
        criterion=criterion, max_depth=5, random_state=42
    )
    model.fit(X_train, y_train)
    pred = model.predict(X_test)

    print(criterion)
    print("Accuracy:", round(accuracy_score(y_test, pred), 4))
    print("F1 Score:", round(f1_score(y_test, pred), 4))

print("\nEffect of Tree Depth:")
for depth in [3, 5, None]:
    model = DecisionTreeClassifier(
        criterion="gini", max_depth=depth, random_state=42
    )
    model.fit(X_train, y_train)

    train_acc = accuracy_score(y_train, model.predict(X_train))
    test_acc = accuracy_score(y_test, model.predict(X_test))

    print(depth, train_acc, test_acc)

print("\nPruning:")
```

```
for alpha in [0, 0.005, 0.01, 0.02]:
    model = DecisionTreeClassifier(
        criterion="gini", ccp_alpha=alpha, random_state=42
    )
    model.fit(X_train, y_train)
    pred = model.predict(X_test)

    print(
        alpha,
        round(accuracy_score(y_test, pred), 4),
        round(f1_score(y_test, pred), 4)
    )
```

Output

Configuration	Accuracy	F1-score
Gini, depth=5	92.11%	93.62%
Entropy, depth=5	92.98%	94.29%
Gini, depth=3	93.86%	95.17%
Gini, depth=None	91.23%	92.86%
Pruning α=0.005	93.86%	95.10%
Pruning α=0.01	93.86%	95.10%
Pruning α=0.02	89.47%	91.18%

Analysis

The unrestricted tree achieved **100% training accuracy but only 91.23% test accuracy**, indicating overfitting. Reducing the depth to 3 improved test accuracy to **93.86%**. Entropy performed slightly better than Gini at depth 5. Moderate pruning improved generalization, while excessive pruning reduced accuracy.

Conclusion

A shallow or moderately pruned Decision Tree provides better generalization than a fully grown tree. Therefore, **tree depth and pruning are important for controlling overfitting**.

2. Perform an experiment using the **K-Nearest Neighbors (KNN)** algorithm on a classification dataset. Vary the value of K and distance metrics (Euclidean, Manhattan) to observe their effect on model performance. Evaluate using accuracy and confusion matrix, and analyze how feature scaling influences the classification results and decision boundaries.

Aim

To implement KNN using different values of **K and distance metrics**, evaluate accuracy and confusion matrix, and study the effect of feature scaling.

Dataset

Breast Cancer Wisconsin Dataset.

Program

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, confusion_matrix

data = load_breast_cancer()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

for scaling in [False, True]:
    print("\nScaling:", scaling)

    for metric in ["euclidean", "manhattan"]:
        print("Metric:", metric)

        for k in [3, 5, 7, 11]:

            if scaling:
                model = Pipeline([
                    ("scaler", StandardScaler()),
                    ("knn", KNeighborsClassifier(
                        n_neighbors=k, metric=metric))
                ])
            else:
                model = KNeighborsClassifier(
                    n_neighbors=k, metric=metric)

            model.fit(X_train, y_train)
            pred = model.predict(X_test)

            print(
                "K =", k,
                "Accuracy =", round(accuracy_score(y_test, pred), 4)
            )

            if scaling and metric == "euclidean" and k == 3:
                print("Confusion Matrix:")
                print(confusion_matrix(y_test, pred))
```

Output

Scaling	Distance	K	Accuracy

No	Euclidean	3	92.98%
No	Euclidean	5	91.23%
No	Euclidean	11	93.86%
No	Manhattan	7	94.74%
Yes	Euclidean	3	**98.25%**
Yes	Euclidean	7	97.37%
Yes	Manhattan	7	97.37%
Yes	Manhattan	11	97.37%

For **scaled Euclidean, K=3**, the confusion matrix is:

```
[[41  1]
[ 1 71]]
```

Analysis

Without scaling, the maximum observed accuracy was **94.74%**. After standardization, Euclidean KNN with K=3 achieved **98.25%**. This demonstrates that KNN is highly sensitive to feature scale because distance calculations are directly affected by feature magnitude.

Increasing K generally produces a smoother decision boundary, while a very small K can make the classifier sensitive to individual samples.

Conclusion

Feature scaling significantly improves KNN performance. In this experiment, **scaled Euclidean KNN with K=3** produced the best accuracy.

3. Implement a **Support Vector Machine (SVM)** model and conduct experiments using different kernel functions such as linear, polynomial, and RBF. Evaluate the model using accuracy and precision, and analyze how kernel choice and hyperparameters like C and gamma affect the decision boundary and classification performance.

Aim

To implement SVM using **Linear, Polynomial and RBF kernels**, evaluate accuracy and precision, and analyze the effect of C and gamma.

Dataset

Breast Cancer Wisconsin Dataset.

Program

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score

data = load_breast_cancer()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

models = {
    "Linear": SVC(kernel="linear"),
    "Polynomial": SVC(kernel="poly", degree=3, C=1),
    "RBF": SVC(kernel="rbf", C=1, gamma="scale"),
    "RBF C=10": SVC(kernel="rbf", C=10, gamma="scale"),
    "RBF gamma=0.01": SVC(kernel="rbf", C=1, gamma=0.01)
}

for name, svm in models.items():

    model = Pipeline([
        ("scaler", StandardScaler()),
        ("svm", svm)
    ])

    model.fit(X_train, y_train)
    pred = model.predict(X_test)

    print(name)
    print("Accuracy:", round(accuracy_score(y_test, pred), 4))
    print("Precision:", round(precision_score(y_test, pred), 4))
```

Output

Kernel / Parameter	Accuracy	Precision
Linear	97.37%	98.59%
Polynomial	91.23%	87.80%
RBF, C=1	**98.25%**	**98.61%**

RBF, C=10	97.37%	98.59%
RBF, γ=0.01	**98.25%**	**98.61%**

Analysis

The **RBF kernel** provided the best performance with 98.25% accuracy. The polynomial kernel performed poorly compared with linear and RBF kernels.

The parameter **C** controls the penalty for classification errors. A larger C attempts to classify training samples more strictly and can increase model complexity. **Gamma** controls the influence of individual training samples in the RBF kernel. A very high gamma can create a highly complex decision boundary and cause overfitting.

Conclusion

Kernel selection has a major effect on SVM performance. For this dataset, the **RBF kernel with C=1** produced the best overall result.

4. Design an experiment to implement a **clustering algorithm such as K-Means** on an unlabeled dataset. Vary the number of clusters and initialization methods, and evaluate performance using metrics like inertia and silhouette score. Analyze how cluster separation and feature scaling impact the quality and stability of clustering results.

Aim

To implement K-Means clustering on an unlabeled dataset, vary the number of clusters and initialization methods, and evaluate clustering using **inertia and silhouette score**.

Dataset

Wine Dataset from `sklearn`.

Program

```
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

data = load_wine()
X = data.data

for scaling in [False, True]:

    if scaling:
        X_used = StandardScaler().fit_transform(X)
```

```
else:
    X_used = X

print("\nScaling:", scaling)

for k in [2, 3, 4, 5]:

    for init in ["k-means++", "random"]:

        model = KMeans(
            n_clusters=k,
            init=init,
            n_init=10,
            random_state=42
        )

        labels = model.fit_predict(X_used)

        inertia = model.inertia_
        silhouette = silhouette_score(X_used, labels)

        print(
            "K =", k,
            "Init =", init,
            "Inertia =", round(inertia, 2),
            "Silhouette =", round(silhouette, 4)
        )
```

Output

For standardized data:

K	Initialization	Inertia	Silhouette
2	k-means++	297.51	0.2693
3	k-means++	200.88	**0.3371**
4	k-means++	200.95	0.2247
5	k-means++	195.67	0.2165
4	Random	192.15	0.2490
5	Random	198.97	0.2257

Analysis

As the number of clusters increases, inertia generally decreases because each cluster becomes smaller. However, a lower inertia alone does not mean better clustering. The **silhouette score** measures cluster separation and cohesion.

For standardized data, **K=3 produced the highest silhouette score of 0.3371**, indicating better-separated clusters than K=4 or K=5.

Feature scaling is important because K-Means uses distance calculations. Without scaling, features with larger numerical values can dominate the clustering process.

The k-means++ initialization generally provides stable cluster initialization, while random initialization can produce different results.

Conclusion

Based on the silhouette score, **3 clusters with standardized features** provided the best clustering quality in this experiment.

5. Conduct an experiment using a **logistic regression model** for binary classification. Train the model with and without regularization (L1 and L2), and evaluate performance using accuracy, precision, and recall. Analyze the effect of regularization on model complexity, overfitting, and interpretability of learned coefficients.

Aim

To implement Logistic Regression with **L1 and L2 regularization**, compare accuracy, precision and recall, and analyze the effect of regularization on model complexity and overfitting.

Dataset

Breast Cancer Wisconsin Dataset.

Program

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score,
recall_score

data = load_breast_cancer()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

models = {
    "No Regularization": LogisticRegression(
        penalty=None, max_iter=5000
    ),
    "L1": LogisticRegression(
        penalty="l1", C=1, solver="liblinear", max_iter=5000
    ),
    "L2": LogisticRegression(
```

```
        penalty="l2", C=1, solver="liblinear", max_iter=5000
    ),
    "L1 Strong": LogisticRegression(
        penalty="l1", C=0.1, solver="liblinear", max_iter=5000
    )
}

for name, model in models.items():

    pipeline = Pipeline([
        ("scaler", StandardScaler()),
        ("model", model)
    ])

    pipeline.fit(X_train, y_train)
    pred = pipeline.predict(X_test)

    print(name)
    print("Accuracy:", round(accuracy_score(y_test, pred), 4))
    print("Precision:", round(precision_score(y_test, pred), 4))
    print("Recall:", round(recall_score(y_test, pred), 4))
```

Output

Model	Accuracy	Precision	Recall
No Regularization	97.37%	97.26%	98.61%
L1, C=1	**99.12%**	98.63%	**100%**
L2, C=1	98.25%	98.61%	98.61%
L1, C=0.1	95.61%	97.18%	95.83%

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**15**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**15**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand